

Coding Test Algorithm

2020/12/27 ~

박지원

서론

백준도 많이 풀고 , 알고리즘 관련 강의도 많이 들었고 , 공부도 열심히 해서 실력이 많이 증진된 것은 엄연히 사실입니다. 하지만 조금만 어려운 수준의 알고리즘이 나오면 많이 풀어봤음에도 개념이 약하고 , 그 때 그 때 풀이가 다르므로 , 오랜 시간 똑같은 고민을 하고 문제를 풀게 됩니다. 이를 해결하고자 기본 알고리즘에 대한 개념과 지금 까지 배운 지식 및 자주 실수하는 부분을 이 노트에 모두 정리합니다.

- 중요한 기초문법(알고리즘을 풀 때 자주 틀리는 문법)
- 컴퓨터수학(이산수학) 과목에서 배운 내용 정리
- 문제해결 필기 노트 정리
- 에버노트에 필기한 실수들 정리
- 종만북 읽고 정리
- 인공지능에서 search algorithm 내용 정리

목차

0강_문제 해결

1강_기초개념

2강_자주하는 실수

~~3강_자료구조 (독립적으로 노트 생성)~~

4강_Dynamic Programming (DP)

5강_Greedy Algorithm

6강_Graph 기초

7강_Graph 심화

8강_문자열

부록_기타 공부 자료

<0강_문제 해결>

1. 스스로 정립한 코딩 테스트 문제 해결 과정

*** 반드시 노트를 준비합니다. ***

1) 문제를 정독합니다.

- 언제나 시간의 압박으로 인해 문제를 제대로 읽지 못합니다. 하지만 문제를 제대로 읽지 않고 풀다가 오류를 찾았을 때 시간이 더 소비된다는 것을 누구보다 잘 알고 있습니다. 반드시 문제를 정독합니다.

2) 문제 해결 방식을 리스트 형식으로 노트에서 설계하기

- 노트에서 번호를 매기면서 문제를 풀어봅니다.
- 로직을 10분 이상 고민해도 생각해내지 못하는 경우 실제 시험이라면 다음 문제로 넘어갑니다.

3) 풀이방법에서 다음 3가지를 검증해봅니다. (위의 2단계와 병행하는 것이 좋습니다.)

- ① 자료 구조 선택 : 시간 복잡도를 고려할 때 가장 적은 시간이 걸리는 자료구조를 선택합니다.
- ② 시간 복잡도 검사 : 제출 결과가 시간초과면 답이 없습니다.
- ③ 공간 복잡도 검사 : 코딩테스트 수준에서는 어지간하면 메모리는 넉넉합니다.

4) 확신하기

- 1) ~ 3) 과정을 본인 역량 하에 완벽하게 수행했다는 가정 하에 , 본인이 스스로 확신을 내립니다. 이 확신을 믿고 문제를 구현하는 도중 어렵고 화가 나서 포기하고 싶은 구현(자주 느끼는 감정)이 나와도 계속 진행합니다.

5) 프로그램으로 구현합니다. (구현 문제를 많이 풀어서 능력을 향상 시킵니다.)

- ① 정답 : Good
- ② 틀림 OR 시간초과 : 나의 역량 부족이므로 실제 시험이라면 다음 문제로 넘어갑니다.

2. 시간복잡도

순위	명칭
$O(1)$	상수 시간(Constant time)
$O(\log N)$	로그 시간(Log time)
$O(N)$	선형 시간
$O(N \log N)$	로그 선형 시간
$O(N^2)$	이차 시간

$O(\log n)$: 로그 시간 알고리즘 (logarithmic time algorithm)
- 로그 함수 형태 알고리즘 (밑이 2인 로그함수 : $\log_2$)

- 일반적으로 CPU 기반의 개인 컴퓨터나 채점용 컴퓨터에서 연산 횟수가 5억을 넘어가는 경우:
 - C언어를 기준으로 통상 1 ~ 3초 가량의 시간이 소요됩니다.
- 코딩 테스트 문제에서 시간제한은 통상 1 ~ 5초가량이라는 점에 유의하세요.
 - 혹여 문제에 명시되어 있지 않은 경우 대략 5초 정도라고 생각하고 문제를 푸는 것이 합리적입니다.

시간제한이 1초인 문제를 만났을 때, 일반적인 기준은 다음과 같습니다.

- N의 범위가 500인 경우: 시간 복잡도가 $O(N^3)$ 인 알고리즘을 설계하면 문제를 풀 수 있습니다.
 - N의 범위가 2,000인 경우: 시간 복잡도가 $O(N^2)$ 인 알고리즘을 설계하면 문제를 풀 수 있습니다.
 - N의 범위가 100,000인 경우: 시간 복잡도가 $O(N \log N)$ 인 알고리즘을 설계하면 문제를 풀 수 있습니다.
 - N의 범위가 10,000,000인 경우: 시간 복잡도가 $O(N)$ 인 알고리즘을 설계하면 문제를 풀 수 있습니다.
- 구종만 선생님은 시간복잡도가 1000만 일 때 가장 안전하다고 생각하십니다.

3. 테스트 케이스 실패 OR 틀렸습니다.

1) 눈으로 천천히 코드를 디버깅해봅니다.

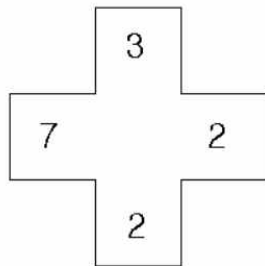
2) 이론이 맞는 것 같다면 강력한 테스트 케이스를 만들어서 다시 디버깅해봅니다.

3) 이제는 이론이 틀렸을 가능성이 높으므로 다시 노트로 돌아와 문제의 이론을 검토합니다.

4) 최후의 방법으로 문제를 다시 풀니다.

5) 다시 풀어도 계속 틀린다면 구글링으로 해결합니다.

<나의 지식이 아닌 , 문제에서 주어진 조건을 이용해서 문제를 해결해야 합니다.>



위와 같은 십자모양의 한 장의 카드에서, 네 모서리에 1 이상 9 이하의 숫자가 하나씩 씌여 있다. 이 네 개의 숫자 중에는 같은 숫자도 있을 수 있다.

모든 가능한 십자 카드가 주어질 때, 각각의 카드는 다음과 같은 '시계수'라는 번호를 가진다. 시계수는 카드의 숫자들을 시계 방향으로 읽어서 만들어지는 네 자리 수들 중에서 가장 작은 수이다. 위 그림의 카드는 시계방향으로 3227, 2273, 2732, 7322로 읽을 수 있으므로, 이 카드의 시계수는 가장 작은 수인 2273이다.

- 네 자리수가 abcd 꼴로 이루어져 있다고 하겠습니다.

① 나의 지식

= 문제를 해석하고 , 스스로 시계수는 'a <= b <= c <= d' 형식이라고 확신하고 문제를 풀었습니다.

② 문제에서 주어진 조건

= 그러나 문제를 자세히 읽어보면 완벽하게 시계수의 조건이 5줄에 걸쳐서 적혀있습니다.

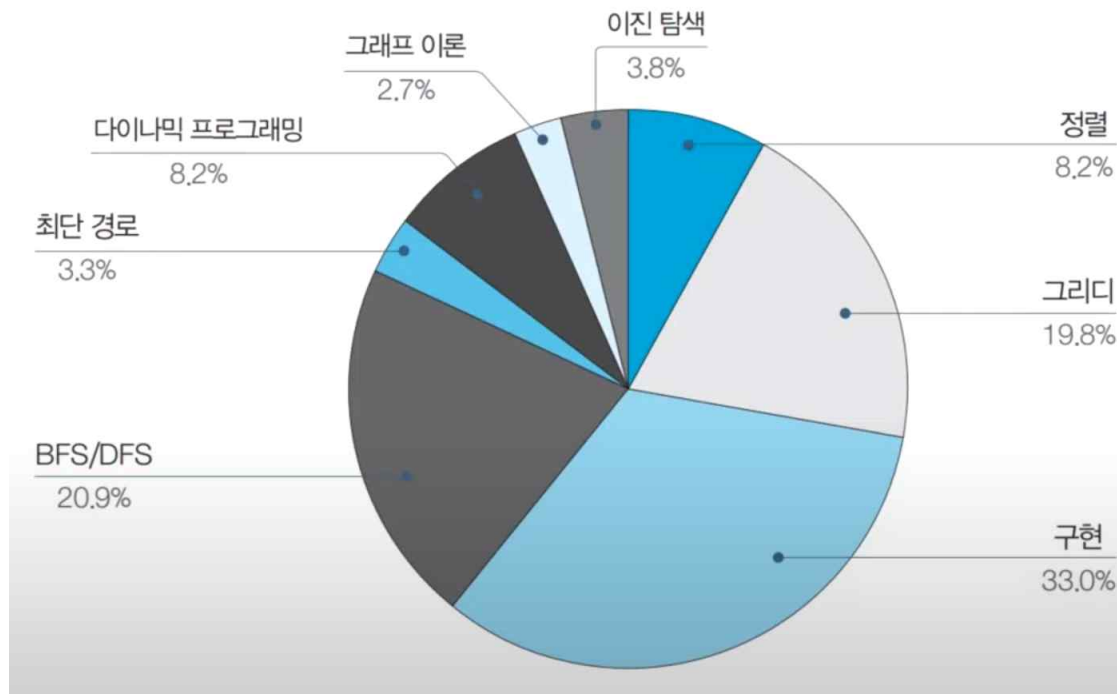
- 결과적으로 나의 지식은 '1132'라는 반례를 갖게 되어 틀렸습니다.

4. 코딩테스트를 경험하며 배운 점

- 코딩테스트는 무조건 난이도 순이 아닙니다.
- **무조건 모든 문제를 훑어보고 , 어떤 유형인지는 파악해야 합니다.**
- 코드를 노트에서 구현할 때부터 확신(어떻게 구현할지 , 시간초과까지 모두 고려했을 때 할 수 있음)이 드는 문제라면 풀고 , 그렇지 않은 문제라면 넘어갑시다.
- 시간초과가 나면 구현하는 방식이 틀린 것이므로 다른 문제로 넘어갑시다.
- 부분 점수와 효율성을 신경 쓰지 말고 다른 문제를 풀 생각을 합시다.
- **$n\log n$** 시간 복잡도는 매우 작습니다. 예를 들어 n 이 100만이어도 $\log n$ 은 20밖에 되지 않습니다.
- 복잡한 구현 문제는 노트에서 충분한 시간을 쏟고 , 확신이 생길 때 구현을 시작하고 의심하지 않고 나아갑니다. 결국 문제에서 구현 방식을 길게 서술해줍니다.
- **자만하고 , 흥분하면 확신 단계를 건너뛰고 바로 구현에 들어가는데 , 절대로 좋지 않은 습관입니다.**

5. 최근 5개년 코딩 테스트 출제 유형

알고리즘 코딩 테스트 유형 분석



<1강_기초개념>

1. memset , memcpy

- `memset(arr, -1, sizeof(arr));`
- `memcpy(new_arr, old_arr, sizeof(arr));`

```
char arr[20];
int arr2[20];
char arr3[20];

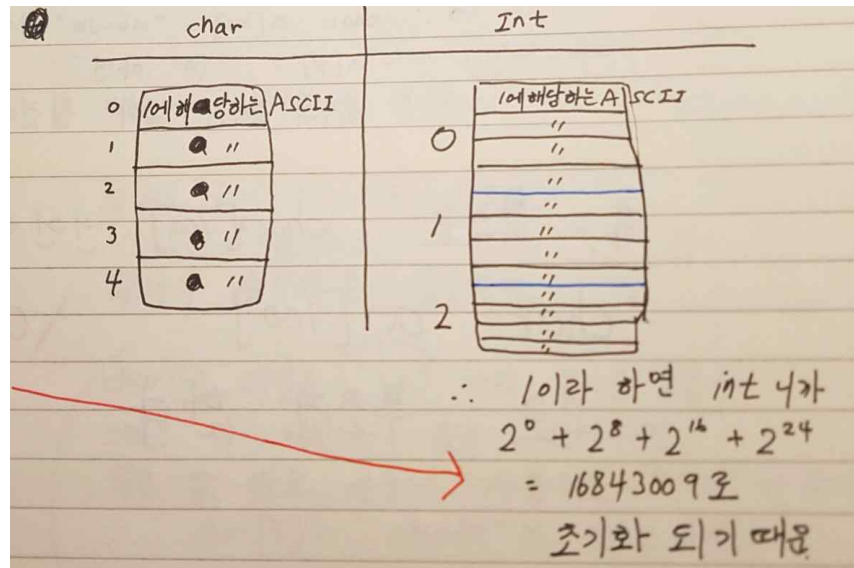
memset(arr, 'a', sizeof(arr));
cout << arr[7]; //a

memset(arr2, 1, sizeof(arr2));
cout << arr2[7]; //16843009

memset(arr2, 0, sizeof(arr2));
cout << arr2[7]; //0

memset(arr2, -1, sizeof(arr2));
cout << arr2[7]; //-1

memcpy(arr3, arr, sizeof(arr));
cout << arr3[7]; //a
```



- memset은 보통 문자열을 초기화할 때 사용하지만, int형 배열도 0 과 -1은 초기화가 가능합니다.
- memcpy는 char 자료형이지만 int형에서의 Copy도 지장이 없습니다.
- memset과 memcpy 모두 pair와 struct 에서도 정상적으로 동작합니다.

```

✓ struct str
{
    int a;
    int b;
    int c;
};

pair<int,int> arr[5];
str arr2[5];
pair<int,int> arr3[5];
str arr4[5];

```

```

memset(arr,-1,sizeof(arr));
for(auto i : arr)
{
    cout << i.first << i.second << " ";
}
cout << "\n";

memset(arr2,-1,sizeof(arr2));
for(auto i : arr2)
{
    cout << i.a << i.b << i.c << " ";
}
cout << "\n";

```

```

memcpy(arr3,arr,sizeof(arr3));
for(auto i : arr3)
{
    cout << i.first << i.second << " ";
}
cout << "\n";

memcpy(arr4,arr2,sizeof(arr4));
for(auto i : arr4)
{
    cout << i.a << i.b << i.c << " ";
}

```

```

$ ./a.exe
-1-1 -1-1 -1-1 -1-1 -1-1
-1-1-1 -1-1-1 -1-1-1 -1-1-1 -1-1-1
-1-1 -1-1 -1-1 -1-1 -1-1
-1-1-1 -1-1-1 -1-1-1 -1-1-1 -1-1-1

```

2. Return값이 존재하는 재귀함수

- 어떠한 상황에서도 조건문의 모든 조건에서 결과로 return값을 도출해야 합니다.
- 재귀 함수는 기저 사례(더 이상 쪼개지지 않는 최소의 작업)에 도달했을 때 반드시 답을 곧장 반환하는 조건문을 포함해야 합니다.
- 일반적인 함수의 return 값은 보통 'return value' 형태이지만 , 재귀함수에서는 재귀함수 자체를 return함으로써 재귀함수를 진행합니다.

ex) return recursive(a)

1) 잘못 작성한 재귀함수

= else 구문에서 Return을 하지 않습니다. 그러므로 모든 경우에서 return 값이 있어야 하는 성질에 위배 됩니다.

```
int find_ancestor(int node)
{
    //i am ancestor
    if (p[node] == node)
    {
        return p[node];
    }
    else
    {
        find_ancestor(p[node]);
    }
}
```

런타임 에
러 (WithoutReturning)

2) 올바르게 작성한 재귀함수

```
int recursive(int n)
{
    //구하고자 하는 값
    if (n == 1)
    {
        return 1;
    }
    //구하고자 하는 값이 아니면 자기자신을 호출한다(재귀).
    else
    {
        return n + recursive(n - 1);
    }
}

int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    cout << recursive(10);
    return 0;
}
```

- 재귀함수를 return 하면 재귀함수를 모두 계산하고 return 합니다.

3. using ll = long long;

The syntax `using ll = long long;` is more modern.

For a simple type definition it does exactly the same as `typedef long long ll;`

But `using` supports template parameters, so the concept is more powerful. In example the following definition does not work good with typedef.

4. 공백포함 문자열 받기

```
string str;  
getline(cin, str);
```

5. Auto

= 변수의 자료형을 초기화 값에 따라 알아서 컴파일러가 자료형을 정해주는 키워드입니다.

- 알고리즘 고수들의 코드에서 auto를 자주 발견할 수 있었습니다.
- Range-based for loop에서 매우 용이하게 쓰입니다.

auto를 사용하면 아래 예제 처럼 변수를 선언할 수 있습니다.

```
auto a1 = 10;           // int 타입  
auto a2 = 10.0f;        // float 타입  
auto a3 = "c";          // char 타입  
auto a4 = "BlockDMask"; // string 타입  
auto a5 = {10, 20, 30}; //int 배열 타입
```

참고 문헌 : <https://blockdmask.tistory.com/384>

<주의 사항>

- auto는 함수의 매개변수로 사용할 수 없습니다.
- auto는 struct의 멤버변수로 사용할 수 없습니다.

6. Range-based for loop

= C++ 11부터 지원하며, 매우 간단하게 for문을 사용하는 방법입니다.

- 일반 배열, vector, set, map 등의 stl에서 모두 사용가능 합니다. 그러나 stack, queue 에서는 사용할 수 없습니다.

1) Vector

```
vector<int> v1;
vector<pair<int,int>> v2;

//input
for(int i=0; i<5; i++)
{
    v1.push_back(i+100);
    v2.push_back({i,i+100});
}

//1. normal vector using Range-based for loop
cout << "1. normal vector using Range-based for loop : ";
for(auto i : v1)
{
    cout << i << " // ";
}
cout << "\n";

//2. pair vector using Range-based for loop
cout << "2. pair vector using Range-based for loop : ";
for(auto i : v2)
{
    cout << i.first << " " << i.second << " // ";
}
return 0;
```

- 출력 부분의 I를 v[i]로 표기하여서 오류를 겪은 경험이 많습니다.

User@DESKTOP-BAGIJS2 ~

\$./a.exe

1. normal vector using Range-based for loop : 100 // 101 // 102 // 103 // 104 //

2. pair vector using Range-based for loop : 0 100 // 1 101 // 2 102 // 3 103 // 4 104 //

2) Map & Vector<pair>

```
vector<pair<int, int>> v;
map<int, int> m;

//input
for (int i = 0; i < 5; i++)
{
    v.push_back({ i, i + 100 });
    m.insert({ i, i + 100 });
}

//map의 3단계 표현
//map 1단계 -> iterator가 있어야합니다.
map<int, int>::iterator iter2;
for (iter2 = m.begin(); iter2 != m.end(); iter2++)
{
    cout << iter2->first << " " << iter2->second << "\n";
}

//map 2단계 -> iterator가 필요 없습니다.
//자료구조 map의 타입은 pair형태 이므로 각원소도 pair형태입니다.
cout << "\n\n";
for (pair<int, int> i : m)
{
    cout << i.first << " " << i.second << "\n";
}
cout << "\n\n";

//map 3단계 -> auto까지 사용하면 더 간편해 집니다.
for (auto i : m)
{
    cout << i.first << " " << i.second << "\n";
}
cout << "\n\n";

//pair
for (auto i : v)
{
    cout << i.first << " " << i.second << "\n";
}
```

3) Range-base For문의 주의사항

우리는 위에서 기본적인 범위기반 for문을 배웠습니다. 하지만,

```
int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for(int elem : arr)
{
    cout << elem << endl;
}
```

이런식으로 위쪽 기본편에서 배운데로 하게 되면, arr 배열에 있는 인자를 elem 이라는 새로운 변수에 복사를 하게 됩니다.

매 반복문이 돌때마다 아래처럼 복사가 됩니다.

```
elem = arr[0];
elem = arr[1];
elem = arr[2];
```

복사를 하게되면 단점이 많습니다.

1. 복사를 했기 때문에 배열의 원래 값을 변경하지 못합니다.
2. 복사를 했기 때문에 복사비용이 발생합니다.

- 탐색(순회)을 하는 경우는 상관없지만 , 값을 변경하는 경우 문제가 발생합니다.

- 기본적인 Range-base For문은 복사를 하는 방식이기 때문에 원래 값을 변경하지 못하므로 참조자를 붙여서 원래 값을 변경시켜야 합니다.

▼ C++ range based for문을 reference로 사용하는 방법.

```
for(int& elem : arr)
{
    elem += 1; //arr[ ] 값도 변경됨.
    cout << elem << endl;
}
```

for(데이터 타입& 변수이름 : 데이터리스트) 와 같이 데이터 타입 뒤에 레퍼런스(&)를 붙이는 방식으로 한다면, 배열의 해당 인자의 값복사가 아닌 변수의 **reference(=참조자, 레퍼런스)**를 가지고 오게 됩니다.

이렇게 할때의 이점을 아래와 같습니다.

1. 복사비용이 들지 않아서 비용이 감소 합니다.
2. 배열의 원래 원소를 변경할 수 있습니다.

```
v.push_back({ 1,2,3 });

//test_1
cout << "TEST_1 : ";
for (int i = 0; i < 5; i++)
{
    for (auto i : v)
    {
        i.a++;
        cout << i.a;
    }
}

cout << "\nTEST_2 : ";
//test_2
for (int i = 0; i < 5; i++)
{
    for (auto& i : v)
    {
        i.a++;
        cout << i.a;
    }
}
```

```
struct str
{
    int a;
    int b;
    int c;
};
vector<str> v;
```

```
TEST_1 : 22222
TEST_2 : 23456
```

① Test_1

- 참조자를 붙이지 않으면 값이 변화하지 않습니다.
- 이 경우 i에 값을 복사하고 i.a만 값이 증가되고 , v[0].a의 값은 변화하지 않습니다.

② Test_2

- 참조자를 붙이면 자료구조 v의 주소를 직접 참조하므로 값이 변합니다.

7. Sort

1) STL의 Sort

C++에서는 STL 함수 중에서 (`#include <algorithm>`를 추가하면 사용할 수 있는) **sort()**를 이용해 정렬을 할 수가 있다.

sort() 함수는 위에서 언급한 quick sort 알고리즘을 바탕으로 만들어졌고 최악의 경우에 대해서도 대비를 해놓은(?) 개선된 quick sort 알고리즘이라고 할 수 있다.

즉 어떠한 경우에서도 $O(n \cdot \log n)$ 을 만족한다!

그렇기에 알고리즘 대회에서 정렬을 필요로 할 때 sort() 함수를 이용하는 것이 가장 좋은 방법이다.

2) 오름차순 Sort

= `sort(v.begin(), v.end())` OR `sort(v.begin(), v.end(), less<자료형>())`

- less가 default이므로 생략해서 오름차순으로 정렬합니다.

- less니까 점점 작아지는 내림차순으로 오해할 수 있으나, 작은 원소부터 나열하겠다는 의미로 less입니다.

ex) 1,3,5,6,7,8,9

3) 내림차순 Sort

= `sort(v.begin(), v.end(), greater<자료형>())`

ex) 9,8,7,6,5,3,1

4) Pair-Sort

= `sort(v.begin(), v.end(), cmp)`

- 3번째 매개변수로 직접 정렬 기준 함수를 만들어서 사용합니다.

<Example>

- 테스트를 위해 2가지 함수 `pair_cmp`와 `struct_cmp`를 정의하였습니다.
- `cmp`함수의 매개변수는 각각의 자료구조에서 사용하고 있는 자료형을 사용하면 됩니다.
- 주석을 보시면 `const`를 사용하고 , 포인터 형 매개변수를 사용합니다. 그로 인해 매개변수의 값을 보호하고 , 메모리의 낭비를 줄일 수 있습니다.
- 알고리즘 문제이므로 메모리가 부족한 상황이 생기면 포인터를 사용합니다.
- 정석은 `return`값으로 '`a.second < b.second`형식'으로 `true` 와 `false`를 나타냅니다. 하지만 개인적으로 `true` , `false`를 직접 `return` 값으로 하는 것이 간단하고 가독성이 좋다고 생각합니다.
- `poj_8979.cpp`가 `struct`에서 금,은,동 메달로 순서 정하는 문제이므로 읽어보면 매우 쉽게 이해할 것 입니다.

```

struct str
{
    string name;
    int ht;
    int kg;
};

vector<pair<string, int>> v;
vector<str> v2;

//bool pair_cmp(const pair<string,int> &a, const pair<string , int> &b)
bool pair_cmp(pair<string,int> a, pair<string , int> b)
{
    if (a.second < b.second)
    {
        return true;
    }
    else
    {
        return false;
    }
}

bool struct_cmp(str a, str b)
{
    if (a.ht < b.ht)
    {
        return true;
    }
    else
    {
        return false;
    }
}

```

```

int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    //input
    v.push_back({ "지원" , 960316 });
    v.push_back({ "지투" , 940316 });
    v.push_back({ "지삼" , 980316 });
    v.push_back({ "지사" , 970316 });
    v.push_back({ "지오" , 910316 });
    v2.push_back({ "지원" , 180 , 85 });
    v2.push_back({ "지투" , 183 , 86 });
    v2.push_back({ "지삼" , 178 , 87 });
    v2.push_back({ "지사" , 199 , 88 });
    v2.push_back({ "지오" , 105 , 89 });
    str obj;
    obj.name = "지육";
    obj.ht = 153;
    obj.kg = 60;
    v2.push_back(obj);

    //pair sort
    sort(v.begin(), v.end(), pair_cmp);
    for (auto i : v)
    {
        cout << i.first << " " << i.second << "\n";
    }

    cout << "\n\n";

    //struct sort
    sort(v2.begin(), v2.end(), struct_cmp);
    for (auto i : v2)
    {
        cout << i.name << " " << i.ht << " " << i.kg << "\n";
    }

    return 0;
}

```

<Runtime ERROR : 등호(==)와 관련된 Strict Weak Ordering>

- 좌측의 코드는 정상적으로 돌아갑니다.
- 우측의 코드는 좌측의 코드와 비교할 때 ' $>$ ' 를 ' $>=$ '로 변경했습니다. 그 결과 Runtime ERROR가 발생하였습니다.

```
bool Cmp(int a , int b)
{
    //a와 b의 값은 무시하고 생각합니다.
    //a > b로 정렬하겠다는 의미는 결국 내림차순(4,3,2,1,0)으로 정렬하겠다는 의미
    //조건문 : 나 왼쪽이 크고 오른쪽이 작은 내림차순으로 정렬할게?
    if (a > b)
    {
        return true; //그래 , 내림차순으로 정렬해
    }
    else
    {
        return false; //안돼 , 그 반대인 오름차순으로 정렬해
    }
}
```



Debug Assertion Failed!

Program: C:\Users\#pjw96\OneDrive\UAA E-,
é#BOJ\Debug\Source.exe
File: C:\Program Files (x86)\Microsoft Visual
Studio\2019\Community\VC\Tools\MSVC\14.29.30037\incl
ude\xutility
Line: 1520

Expression: invalid comparator

For information on how your program can cause an assertion
failure, see the Visual C++ documentation on asserts.

(Press Retry to debug the application)

아마 아래처럼 = 부호를 포함해서 작성하셨을 겁니다.

```
bool cmp(int a, int b) {
    if (a >= b)
        return true;
    return false;
}
```

sort를 위한 compare 함수는 반드시 strict weak ordering을 만족해야 합니다. 즉, $a=b$ 인 경우에는 $a < b$ 도 false이고 $a > b$ 도 false가 되어야 합니다.

8. Iterator

```
set<int> s;  
map<string, int> m;  
set<int>::iterator iter_s;  
map<string, int>::iterator iter_m;
```

```
cout << *s.find(5) << "\n";  
cout << (*m.find("지원")).first << " " << (*m.find("지원")).second << "\n";  
cout << m.find("지원")->first << " " << m.find("지원")->second << "\n";
```

iterator는 자료구조의 원소를 가리키는 포인터입니다.

그러므로 find로 해당 원소를 찾으면 해당 원소의 실제 주소를 가리키고, 찾지 못하면 자료구조의 가장 끝인 end를 가리키게 됩니다.

예전에는 iterator 순회를 통하여 원소를 찾았지만 이제는 범위기반 for문을 이용합니다.

```
for (auto i : m)  
{  
    cout << i.first << " " << i.second << "\n";  
}
```

9. 이진수 표현

- 컴퓨터는 무슨 일이 있어도 0과 1로만 모든 것을 표현합니다.
- 이진수를 사람이 이해하기는 어려우므로 그나마 알아볼 수 있는 16진수를 사용해서 이진수를 표현할 수 있습니다.
- 비트 마스크를 사용할 수 있습니다.

```
char a = 0x30;  
int b = 0xffff;  
  
/*  
8비트 중에 앞에 4개는 0x30이므로 0011 뒤에 4개는 0x00이므로 0000  
문자열에서 0011 0000을 우리는 문자 0으로 약속 합니다.  
*/  
cout << a << endl;  
  
/*  
32비트 중에서 앞에 16개는 0x00이고 뒤에 16개는 4개씩 f이므로  
\n 0000 0000 0000 0000 1111 1111 1111 1111 입니다.  
*/  
cout << b;
```

```
$ ./a.exe  
0  
65535
```

10. 나머지 연산으로 flag 대체하기

- 언제나 나누어 주는 값은 값의 개수 입니다.

<Example : '0 -> 3 , 1 -> 0 , 2 -> 1 , 3 -> 2'를 구현하시오>

= $(n+3) \% 4$

<Example : '3 -> 0 , 0 -> 1 , 1 -> 2 , 2 -> 3'을 구현하시오>

= $(n+1) \% 4$

11. Runtime ERROR (NZEC)

 15684

런타임 에러 (NZEC)

- return 0 대신에 exit(0)를 사용하면 0을 리턴하고 종료하므로 괜찮지만 exit(1)을 사용하면 위의 ERROR가 발생하므로 exit(0)를 이용합니다.

12. for문 범위

for(int i=0; i<-1; i++) : 이 문법도 ERROR 없이 가능합니다. 결국 for문을 아예 수행하지 않습니다.

13. 배열 순회에서 거꾸로 시작하는 방식도 고려해보아야 합니다.

14. 구조체와 다수의 배열

1) 배열의 다양한 자료형

```
pair<int , vector<int>> arr[5][5];
```

```
arr[2][2].second.push_back(11);  
arr[2][2].second.push_back(12);  
arr[2][2].second.push_back(13);  
cout << arr[2][2].second[2];  
$ ./a.exe  
13
```

- 이제는 배열을 만들 때 복잡한 형태의 자료형도 구사할 수 있습니다.
- 배열을 pair로도 사용하고 struct로도 사용하는 것을 배우기는 했지만 지금까지 사용하지 않았습니다. 이제는 실제로 사용할 수 있게 되었습니다.

2) 다수의 배열을 만드는 것 보다 구조체를 만들어서 문제를 해결합니다.

```
struct str  
{  
    double a;  
    int b;  
};  
  
str arr1[5];  
double arr2[5];  
int arr3[5];
```

- 구조체는 int b 까지도 double형으로 간주하여 8byte를 할당합니다. 그러므로 8byte * 2개 * 5(배열의 크기) = 80byte
- 배열을 2개 만들면 1개는 double형이라 40byte고 다른 1개는 int형이라 20byte만 할당합니다.
- 큰 자료형을 포함한 구조체 배열보다는 일반 배열을 다양하게 만드는 것이 메모리의 관점으로는 이점이 있으나 캐시를 생각해보면 연속적으로 저장하는 구조체가 더 빠를 것 입니다. 또한 구현의 가독성에 있어 구조체로 사용하는 것이 더 알아보기 쉽습니다.
- 결론 : 다양한 데이터를 포함한 배열을 만들 때에는 구조체를 사용합시다.

```
cout << sizeof(arr1) << " " << sizeof(arr2) << " " << sizeof(arr3);
```

```
$ ./a.exe  
80 40 20
```

15. 반올림과 소수의 오류

- 문제에서 반올림 된 소수를 요구할 때는 printf와 %.을 이용합니다.
- printf는 기본적으로 반올림이 default이므로 따로 처리하지 않아도 됩니다.

<1.15를 두 짝 자리에서 반올림 했지만 1.1인 오류>

```
double a = 1.14;  
double b = 1.15;  
double c = 1.1500001;  
double d = 1.15000000000000001;
```

```
printf("%.15f\n", d);  
printf("%.1f %.1f %.1f %.1f", a , b , c , d);
```

```
1.150000000000000  
1.1 1.1 1.2 1.2
```

16. 다양한 케이스로 나뉘는 경우의 문제

- 분기를 4개 이상 나누어서 하는 방식은 귀찮고, 이게 맞는지 의심을 하는 경우가 많습니다.
- 이런 경우가 정말 노트에서 검증을 많이 하고, 빠르게 구현해야 하는 문제입니다.

17. 올림피아드 느낌의 수학 문제 알고리즘 접근 방법

- 수학적인 문제가 나올 때, 답이 보이지 않는다면 과감히 넘어가고 다른 문제부터 풀니다. 그리고 다시 돌아와서 작은 수를 예시로 하나 만들고, 해당 문제를 풀어봅니다.
- 예를 들어 1~10억까지 3의 개수가 문제라면, 1 ~ 5326정도 까지 3의 개수를 풀어봅니다. 그러다보면 규칙이 보일 수 있습니다.

<2강_자주 하는 실수>

1. for문 순서

```
//초기값 , 조건식 , 증감식
for (int i = 0; i < 10; i++)
{
    //statement
}

//for문을 빠져나오면 i= 9+1 이므로 10이된다.
```

1) 초기 값

2) 조건에 맞는지 검사

3) True -> statement 실행

4) 증감 식은 i++ OR ++i 상관 없습니다.

5) False 일 때 까지 statement 실행

- For문에서 빠져나올 때 증가하는 실수가 자주 발생하므로 , For문에서 사용하는 iterator는 어지간하면 지역변수로 사용합시다.

2. 역 방향 for문

= 슬프지만 너무 자주 하는 실수입니다.

- `for(int i=10; i>=0; i--)`를 맨날 `for(int i=10; i>=0; i++)`로 쓰고 있습니다.

3. printf 와 cout을 혼용하면 안 됩니다.

= 먼저 printf로 출력을 하고 그 다음에 cout으로 출력을 하는 경우에도 cout이 먼저 출력되기 때문에 원하는 출력 결과가 나오지 않습니다.

4. 찾기 어려운 Overflow

= 자료형이 int를 넘어가는 문제는 모든 자료형을 long long으로 하면 틀릴 걱정이 없습니다.

```
int main()
{
    int a = 12312323;
    int b = 12312323;
    long long answer_1 = a*b;
    printf("Using Int : %d\n" , answer_1);

    long long c = 12312323;
    long long d = 12312323;
    long long answer_2 = c*d;
    printf("Using Long Long : %ld\n" , answer_2);
}
```

pjw96@ParkJiWon ~/BOJ/source
\$./a.exe
Using Int : -1868023287
Using Long Long : 151593297656329

5. 긴 소스코드에서 전역변수와 지역변수

= 변수 이름을 다양하게 쓰지 않다보니 소스코드의 길이가 길어진다면 겹치는 경우가 필연적으로 발생합니다. 이 때 전역변수와 지역변수를 동일하게 쓰는 경우가 발생하고 , 지역변수의 우선순위가 높아 전역변수의 값이 함수내에서 변하지 않을 수 있으므로 주의해야 합니다.

6. 0-1 offset

<Example : 배열의 0번째 원소와 1번째 원소가 같을 때 배열의 0번째부터 10번째 까지 모두 같은지 판별하시오>

- 시간복잡도 관점에서 생각해보면 1번째 원소와 2번째 ~ 10번째 원소를 비교하면 됩니다.
- 하지만 0-1 offset 문제가 발생할 수 있으므로 간단하게 0번째 원소와 0번 ~ 10번 까지 모두 비교합시다.

7. 익숙한 것을 문제에서 바꾸면 디버깅 하는데 너무 오랜 시간이 걸립니다.

- 익숙하기 때문에 문제를 제대로 읽지 않고 무의식으로 습관성 코딩을 하고 , 눈으로 디버깅을 해도 절대 금방 찾지 못합니다.

ex) 문제에서 열 , 행으로 입력 값을 주어주는 경우

ex) Path를 만들 때 항상 north가 0인데 문제에서는 south가 0인 경우

8. 이게 무한루프가 나옵니다.

```
vector<int> v;  
for(auto i=0; i>=v.size(); i--)  
{  
    cout <<"z";  
}
```

- v.size()는 unsigned 형이므로 i는 unsigned이므로 음수가 될 수 없으므로 i-1은 -1이 아니라 오버플로우가 일어나 매우 큰수가 됩니다. 그러므로 항상 0보다 크므로 무한루프가 발생합니다.

9. 어떤 값을 10007로 나눈 값을 구하시오

- 문제를 풀다 보면 값이 너무 커서 결과 값을 어떠한 수로 나누는 문제가 종종 출제 됩니다. 간단하게 생각하면 결과 값을 구하고 10007로 나누면 될 것 같지만 unsigned long long을 사용해도 결과 값을 나타낼 수 없는 경우가 발생할 수 있습니다. 그러므로 **중간 중간의 계산마다 10007로 나누어 줍시다.**
- 이런 문제 유형에서 $10007 + 1 = (10007 * 10^9) + 1$ 입니다.

10. cout 출력

1) 분수와 소수

```
cout << 3 / 2 << "\n"; // 1
cout << 3.0 / 2.0 << "\n"; // 1.5
cout << 1.5 << "\n"; // 1.5

// *** 나쁜 코드 ***
cout << 3.0 / 2 << "\n"; // 1.5
```

<중요한 개념>

- 기본적으로 정수는 Int 취급, 소수는 Double 취급을 합니다.
- 정수로 이루어진 분수는 Int로 취급하므로 매우 위험합니다.
- 소수로 이루어진 분수는 Double로 취급하므로 안전합니다.
- 정수와 소수가 혼합되어 있는 분수는 다른 type 연산이므로 사용하지 맙시다.

2) 형 변환

```
cout << (double)(3 / 2) << "\n"; 1
cout << (double)3 / (double)2; 1.5
```

- 3/2 연산을 먼저하고 double로 강제 형 변환(=cast)을 시키므로 (double) 1 과 같습니다.
- 아래의 코드처럼 작성해야 올바른 결과가 나오며, 보통은 소수를 이용합니다.

11. char to int는 익숙하나, int to char가 나올 때 실수하는 경우

- char to int는 char의 값에서 -'0'을 하면 됩니다.
- int to char를 하는 경우에도 -'0'을 해서 당황한 적이 있습니다. 이 경우는 반대로 +'0'을 해주어야 합니다.

<4강_동적 계획법 (DP)>

*** 노트에서 죽어도 생각이 나지 않아 포기가 하고 싶은 유형이지만 , 생각이 나는 순간 구현은 매우 쉽기 때문에 포기하지 맙시다. ***

1. Bottom-Up 방식 : 점화식 + Memoization

- 문제의 크기를 직관적으로 답을 구할 수 있을 수준의 **작은 문제로 변형**합니다.
- 작은 문제의 답을 dp 배열에 기록하는 memoization을 진행합니다.
- dp 배열에 저장된 값을 이용하여 점점 더 큰 문제로 올라갑니다. 그러다보면 최종적으로 문제의 답을 찾게 됩니다.
- **점화식(규칙)을 찾는 것이 핵심**입니다.
- 개발자가 어려운 버그나 구현을 만났을 때 프로토타이핑을 통해 해결하는 것과 맥락은 비슷하다고 생각합니다.
- **dp[1], dp[2], dp[3] ... 처럼 몇 가지를 직관적으로 구해보고, 그림을 그리는 방식**이 문제 해결에 도움이 됩니다.
- Top-Down 보다 제가 선호하는 방식입니다.
- 새로운 값을 만드는 것이 아니라 , 기존의 값들을 재활용하기 때문에 시간복잡도가 매우 감소합니다.

ex) 크기가 100인 배열에서 어떤 것을 물어보는 문제라면 크기가 1일 때의 답을 구해보고 , 크기가 2일 때의 답을 구해보고 , ... , 크기가 100일 때의 답을 구합니다.

2. Top-Down 방식 : 재귀 함수 + Memoization

재귀 함수를 이용한 풀이

```
1  #include <stdio.h>
2  long long fibo(int n);
3
4  int main(){
5      printf("%lld", fibo(10));
6      return 0;
7  }
8
9  long long fibo(int n){
10     if (n < 2) return n;
11     return fibo(n-1) + fibo(n-2);
12 }
```

- 재귀적으로 구현하였지만 , 이는 시간복잡도가 매우 높은 방식입니다. 그러므로 memoization을 사용합니다.

3. Memoization

memoization 미국식  영국식  

메모이제이션. 컴퓨터 프로그램이 동일한 계산을 반복해야 할 때, 이전에 계산한 값을 메모리에 저장함으로써 동일한 계산의 반복 수행을 제거하여 프로그램 실행 속도를 빠르게 하는 기술.

- Memoization

1. 재귀/점화식을 이용한 Top-down 방식에 쓰임
2. 동일한 계산을 반복하지 않기 위해,
이미 계산된 값은 배열에 저장해 놓고
다음 방문부터는 그 값을 바로 넘겨줌으로써
중복된 계산을 피한다.

메모이제이션을 이용한 풀이

```
1  #include <stdio.h>
2  long long fibo(int n);
3
4  //배열들 초기값은 0이다
5  long long memo[100] = {1,1};
6
7  int main(){
8      for (int i = 1; i<=60; i++) printf("%lld \n", fibo(i));
9      return 0;
10 }
11
12 long long fibo(int n){
13     if (n < 2) return n;
14     if(memo[n] != 0){ //메모되어있다면
15         return memo[n];
16     } else { //메모안되어있다면
17         memo[n] = fibo(n-1) + fibo(n-2);
18         return memo[n];
19     }
20 }
```


4. 시간을 더 줄일 수 있습니다

- 완전탐색 방식을 DP에서 사용하는 Memoization을 활용한 풀이로 바꾸면 시간복잡도가 많이 줄고 해결되는 문제가 많은 것은 사실입니다. 그리고 Memoization을 활용할 점화식을 찾는 것 자체가 DP의 핵심입니다.
- 하지만 **어려운 DP일수록 Memoization을 통해 줄인 시간복잡도가 여전히 시간초과인 경우가 존재합니다.**

<시간 줄이는 방법>

① DP배열 계산(=점화식)을 거치지 않아도 훨씬 짧은 연산으로 DP배열의 값을 구하는 예외 케이스가 존재할 때 , 해당 연산으로 DP배열의 값을 구합니다.

②

ex) Boj_1699

5. Knapsack 문제

- 애송이 DP문제는 언제나 1차원 배열에서 Memoization을 하면서 풀어 왔습니다.
- 하지만 , 난이도가 증가하는 DP문제는 **2차원 배열**에서 Memoization을 하면서 풀어야 합니다.
- **2차원 배열에는 필요 없는 원소도 존재하지만 , 1차원 배열처럼 모든 원소의 값을 구하고 필요한 원소의 값만 사용합니다.**

$w_i = item_i$ 의 무게

$p_i = item_i$ 의 가치

W = 배낭에 넣을 수 있는 최대 무게

$P[n][W]$ 는 아래 식과 같으므로

$$P[n][W] = \begin{cases} \max(P[n-1][W], p_n + P[n-1][W - w_n]) & \text{if } W \geq w_n \\ P[n-1][W] & \text{if } W < w_n \end{cases}$$

(n-1)번째 행에서는 $P[n-1][W]$ 와 $P[n-1][W - w_n]$ 항만 필요!

						$P[0][W]$
		...	...	...	...	...
		$P[n-1][W - w_n - w_{n-1}]$		$P[n-2][W - w_n]$		$P[n-2][W]$
				$P[n-1][W - w_n]$	...	$P[n-1][W]$
						$P[n][W]$

- Reference_1 : '문제 해결 PPT' : DP_1 (0-1 knapsack Problem)
- Reference_2 : <https://gglifer-cs.tistory.com/24>
- Reference_3 : <https://gsmesie692.tistory.com/113>

<Example_1 : boj_1535>

문제

세준이는 성형수술을 한 후에 병원에 너무 오래 입원해 있었다. 이제 세준이가 병원에 입원한 동안 자기를 생각해준 사람들에게 감사하다고 말할 차례이다.

세준이를 생각해준 사람은 총 N 명이 있다. 사람의 번호는 1번부터 N 번까지 있다. 세준이가 i 번 사람에게 인사를 하면 $L[i]$ 만큼의 체력을 잃고, $J[i]$ 만큼의 기쁨을 얻는다. 세준이는 각각의 사람에게 최대 1번만 말할 수 있다.

세준이의 목표는 주어진 체력내에서 최대한의 기쁨을 느끼는 것이다. 세준이의 체력은 100이고, 기쁨은 0이다. 만약 세준이의 체력이 0이나 음수가 되면, 죽어서 아무런 기쁨을 못 느낀 것이 된다. 세준이가 얻을 수 있는 최대 기쁨을 출력하는 프로그램을 작성하시오.

입력

첫째 줄에 사람의 수 N ($N \leq 20$)이 들어온다. 둘째 줄에는 각각의 사람에게 인사를 할 때, 잃는 체력이 1번 사람부터 순서대로 들어오고, 셋째 줄에는 각각의 사람에게 인사를 할 때, 얻는 기쁨이 1번 사람부터 순서대로 들어온다. 체력과 기쁨은 100보다 작거나 같은 자연수 또는 0이다.

출력

첫째 줄에 세준이가 얻을 수 있는 최대 기쁨을 출력한다.

```
for (int i=1; i<=N; i++) {
    for (int j=1; j<=100; j++) {
        dp[i][j] = dp[i-1][j]; /* i 번째 사람에게 인사를 할 수 없다면 */
        if (j-L[i] > 0) { /* i 번째 사람에게 인사를 할 수 있다면 */
            dp[i][j] = max(dp[i][j], dp[i-1][j-L[i]]+J[i]);
        }
    }
}
```

- 순서를 무시하고 , 그 순간의 모든 경우를 고려해야 합니다.

- **문제 해법** : 3번 사람에게 인사를 하는 경우 1번 , 2번의 다음 순서이므로 1번 , 2번을 이용해서 문제를 풀 것 같지만 그렇지 않습니다. 3번 사람에게 인사를 하는 경우를 계산할 때 , 3번 사람만 생각하고 , 나의 체력이 100가지(1 ~ 100)일 때를 모두 구합니다. 예를 들어 , 3번 사람에게 인사를 하는 경우 체력이 50쯤이면 , 나의 체력이 1~50인 경우는 인사를 하지 않고 , 51~100이면 인사를 합니다.

- 모든 경우를 고려하므로 시간복잡도가 클 것으로 예상되지만 , Memoization을 이용하므로 시간복잡도는 문제가 되지 않습니다.

6. 자주 하는 실수

1) 문제의 정답이 항상 dp 배열의 마지막 원소는 아닙니다.

- 대다수의 DP문제의 해답은 memoization을 한 배열의 마지막 원소입니다.
- 하지만 dp 배열의 어떤 값도 정답이 될 수 있습니다.

ex) dp 배열의 값이 가장 큰 문제가 정답일 때 dp 배열이 “1 , 2 , 5 , 9 , 6” 라면 정답은 9가 됩니다.

reference : <https://www.acmicpc.net/problem/1965>

7. 잡설

1) DP로 접근해야 하는가?

- 실제 시험에서는 해당 문제를 DP로 해결해야 하는지 알 수가 없습니다.
- 일단 , Brute-Force로 접근해보고 시간복잡도가 초과하면 DP로 풀어보자.

<5강_그리디 알고리즘>

1. 정의

= 당장 눈앞에 보이는 최적의 상황만을 쫓는 알고리즘

- 코딩테스트에서의 대부분의 그리디 문제는 그리디 알고리즘으로 얻은 해가 최적의 해가 되는 상황에서 , 이를 추론할 수 있어야 풀리도록 출제됩니다.
- 언제나 optimal 하지는 않지만 , 특정한 상황에서는 언제나 optimal 합니다. 'Ai study 4.1'을 참고하면 Greedy Algorithm에서 optimal 하다고 구한 local maxima는 지역적으로는 optimal 하지만 그것이 문제 전체에서 optimal 한 global maximum은 아닐 수 있습니다.
- Greedy Algorithm의 local maxima가 언제나 global maximum과 동일함을 증명할 수 없다면 , DP나 완전탐색으로 문제를 해결해야 합니다.
- 개인적으로 알고리즘 시험에서 가장 중요한 논리는 이 문제에서 greedy algorithm을 써서 구한 local maxima가 언제나 global maximum임을 증명할 수 있어야 하고 , 만약 증명이 된다면 DP나 완전탐색으로 풀 경우 시간복잡도가 초과될 것 입니다. 그러므로 언제나 어떤 문제를 Greedy Algorithm을 통해 해결하기로 결정했다면 반드시 증명과정을 거쳐야 합니다.
- 극단적(무조건 크거나 , 무조건 작은 경우)으로 문제에 접근하며 정렬 기법과 함께 사용.
- 모든 Greedy Algorithm은 일반적으로 Heuristic Algorithm 입니다.

<6강_그래프 기초>

1. 그래프 표현 방법

1) 인접리스트

= 인접한 정점을 나열합니다.

2) 인접행렬

= 정점_i에서 정점_j로 가는 모서리가 있으면 1, 없으면 0으로 행렬을 만듭니다.

<인접리스트 vs 인접행렬>

- n : 노드의 개수
- m : 하나의 노드에서 최대 차수 (ex : 차수가 4, 3, 2, 2 면 m은 4입니다.)
- 보통 Edge의 개수와 m은 비례합니다. Edge가 많다면 노드에서의 차수들은 당연히 값이 커집니다.
- 인접리스트 = $m * n$ 보다 작거나 같은 개수의 메모리를 사용합니다.
- 인접행렬 = n^2 의 메모리를 사용합니다.

① Edge의 개수가 적은 경우

= m의 값이 작을 것이므로 상대적으로 인접리스트가 메모리를 적게 사용합니다.

② Edge의 개수가 많은 경우

= m의 값이 클 것이므로 상대적으로 인접행렬이 메모리를 적게 사용합니다.

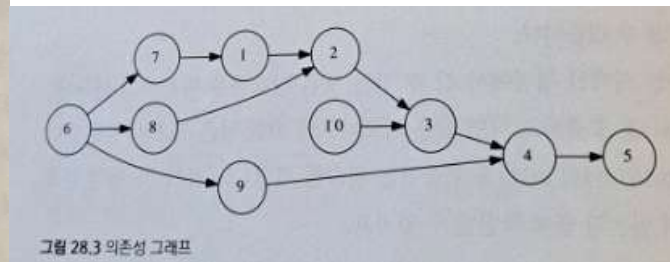
2. Topological Sort

= 위상 정렬

= 그래프가 부분순서 관계 일 때 노드들끼리 순서가 존재해서 노드들을 해당 순서로 정렬하는 방법입니다.

- 공집합이 아닌 집합에서 관계가 부분순서일 때 그래프를 만들고, **그래프에서 극소원소를 선택하고 제외하는 행동을 반복**하여 구합니다. 극소원소는 1개 이상 일 수 있으므로, 그런 상황에서 어떤 극소를 뽑는 것은 프로그래머의 자유이므로 **위상정렬은 다양하게 나올 수 있습니다.**

1. 김치를 다진 마늘과 볶는다.
2. 볶은 김치에 돼지고기, 고춧가루, 다진 마늘을 넣고 더 볶는다.
3. 멸치 육수를 붓고 끓인다.
4. 파와 양파를 넣고 좀더 끓인다.
5. 맛있게 먹는다.
6. 냉장고에서 재료들을 꺼낸다.
7. 김치를 썰다.
8. 돼지고기를 썰다.
9. 파와 양파를 썰다.
10. 멸치 육수를 낸다.



<Example>

부분순서집합 $(\{1, 2, 4, 5, 12, 20\}, |)$ 에 대해 양립할 수 있는 전체순서를 찾아라.

- '|' : 서로 다른 두 수에서 큰 수를 작은 수로 나누었을 때 나머지가 0입니다.
- 1 -> 5 -> 2 -> 4 -> 20 -> 12

*** ai study의 3.3 & 3.4를 읽어 봅시다. ***

3. DFS

1) 3가지 핵심 요소

- ① **재귀함수 & 조건** : 기본적으로 DFS는 재귀함수의 로직을 이용하는 탐색 기법입니다. 재귀함수는 실행 흐름만 이해하면 간단합니다.
 - ② **Visit 배열** : 부모에서 자식으로 탐색을 진행할 때 , 이전에 탐색한 자식을 가면 무한루프가 생성되므로 Visit 배열을 통해 검사합니다.
 - ③ **Index가 Depth인 배열** : 배열의 n번 index는 depth가 n일 때의 노드 값을 저장합니다. depth가 3인 node_A를 탐색할 때 , 이 배열의 0번 index ~ 2번 index는 node_A의 부모와 조상이므로 사용하면 되고 , 여기서 배열의 3번 index 값을 변경합니다.
- 위의 개념은 '4)방문 여부를 저장하는 Visited 배열'에 자세하게 설명해놓았습니다.

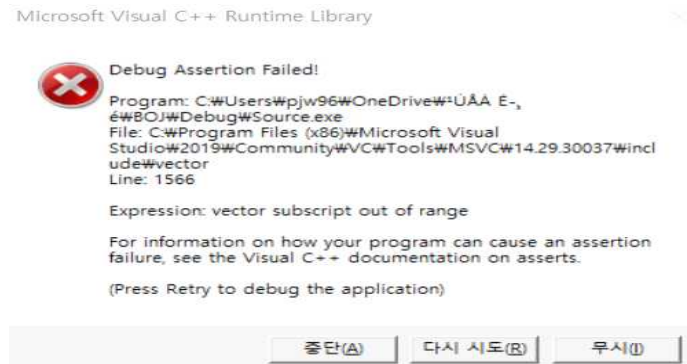
2) DFS 구현에서 어려워하는 부분

① 재귀적 사고

- DFS는 재귀 함수의 형태로 구성되어 있습니다. 그렇기 때문에 BFS와는 다르게 구현의 모양이 쉽지 않고 , 특히 오류를 찾기 어렵습니다.

② Root의 Depth = 0

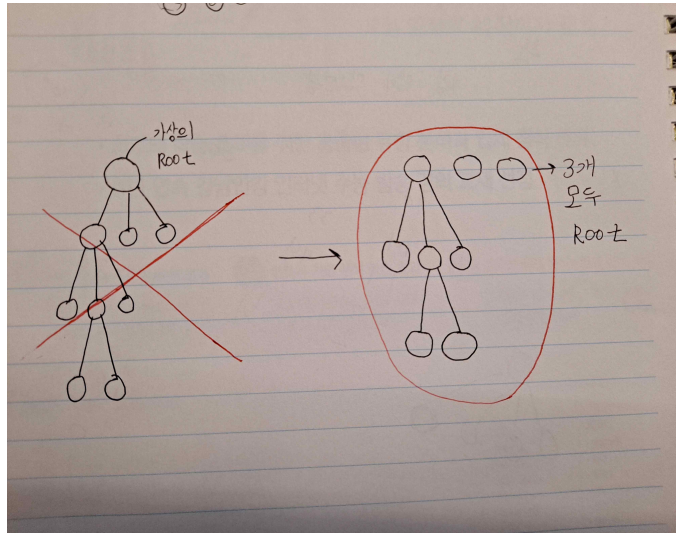
- Vector와 같이 사용하는 경우에 index가 0인 것이 편리하고 , 헛갈리지 않으므로 Root의 Depth를 0으로 구현합니다.
- **Vector의 영역을 벗어나면 아래의 오류가 발생하는데 이를 방지하기 위해 Vector 대신 크기가 정해진 Array를 쓰는 것도 좋은 방법입니다**



<Vector Subscript out of Range>

- Vector로 구현하는 경우 , 'v[depth+1]' 과 같은 코드를 작성하면 좌측의 Vector의 범위를 벗어나는 영역을 참조하여 Runtime ERROR가 발생할 수 있습니다.

③ Root가 여러 개인 경우



- Root Node를 가상의 빈 Node를 생성하고, 아래부터 시작하는 방식을 고수했지만 그럴 필요 없고 Root Node를 여러 개로 시작하면 됩니다.

```
for (int i = 1; i < node + 1; i++)  
{  
    dfs(i, 1);  
}
```

Dfs(0);

① dfs(i,1)

= 1년 전에 푼 풀이로 루트가 여러 개 일 때 for문으로 하나씩 처리합니다.

② Dfs(0)

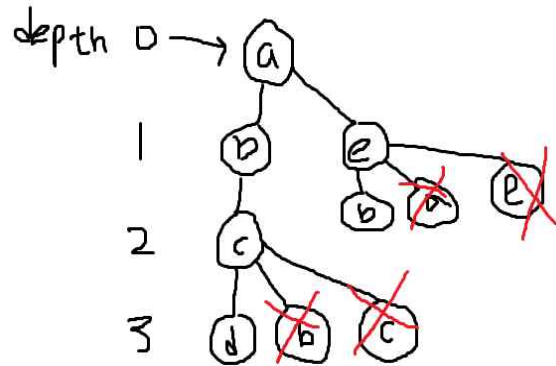
= 루트가 여러 개 일 때 Dfs 내부 로직에서 자체적으로 for문을 통해 다수의 루트를 생성합니다.

④ 방문 여부를 저장하는 Visited 배열

= Depth를 index로 하는 visited 배열을 완벽하게 구현할 수 있습니다.

- Depth 마다의 값을 저장하는 배열입니다.

<Example : 1 ~ 10 까지 수를 일렬로 나열하는데 , 10가지 수가 모두 달라야 합니다.>



```
/*
1. 왼쪽 리프에서 depth 3일 때 검사
- visited의 index 0 , 1 , 2 는 모두 depth
- depth가 3일 때는 depth-1인 2까지 확인해야 하므로
  visited[0] ~ visited[2]를 확인
- depth가 3일 때 d는 가능하나 b , c는 불가능

2. 오른쪽 리프에서 depth 1일 때 검사
- depth가 1일 때 e를 검사해보면 , depth-1이 0이므로
  visited[0]을 확인
- e는 visited[0]이 a이므로 가능함.

3. 오른쪽 리프에서 depth 2일 때 검사
- 모든 논리는 똑같으나 바로 이전의 depth가 1일 때 검사과정에서
  visited[1] = 'e'로 변경 되었습니다.
- depth가 2일 때 b는 가능하나 a , e는 불가능
```

```
//왼쪽 리프에서 depth 3일 때 검사하는 순간의 visited 배열
visited[0] = 'a';
visited[1] = 'b';
visited[2] = 'c';

//오른쪽 리프에서 depth 2일 때 검사하는 순간의 visited 배열
//왼쪽에서 visited[2]가 c였고 , 검사하기를 완료하기 전에는 바뀌지 않습니다.
visited[0] = 'a';
visited[1] = 'e';
visited[2] = 'c' -> 'b';
```

3) Back Tracking

4) 시간복잡도

① 인접 행렬

= $O(V^2)$

- 그래프에 존재하는 모든 노드는 dfs()함수를 반드시 한 번은 호출합니다. 여기서 V개
- dfs()에서는 자신과 인접한 노드가 어떤 노드인지 탐색해야 합니다. 여기서 V개
- 그러므로 $V * V$ 로 인해 V^2 의 시간복잡도 입니다.

② 인접 리스트

= $O(V+E)$

- 그래프에 존재하는 모든 노드는 dfs()함수를 반드시 한 번은 호출합니다. 여기서 V개
- 각각의 노드에서 인접한 노드를 인접 리스트로 이미 구해놓았고 , 모든 노드들에 대해서 연결된 Edge의 개수는 최악의 경우도 $2 * E$ 가 되므로 2배를 생략하여 $V+E$ 가 됩니다.

5) 문제 유형

① 순열과 조합

- 깊이의 최대가 5라고 가정 할 때 , depth가 1부터 시작하면 depth가 6이 아니면 평범하게 for문을 사용하고 , depth가 6일 때 return 하는 형식으로 해결합니다. **깊이의 최대는 보통 전역변수로 사용하여 함수의 매개변수로 전달하지 않습니다.**

```
void Dfs(int idx , int depth) //main에서 Dfs(1,1)로 수행
{
    if(depth > m)
    {
        for(int i=1; i<=m; i++)
        {
            cout << arr[i] << " ";
        }
        cout << "\n";
        return;
    }
    for(int i=idx; i<=n; i++)
    {
        arr[depth] = i;
        Dfs(i+1 , depth+1);
    }
}
```

- 순열과 조합의 정석 입니다.
- 순열은 순서가 중요하지만 조합은 순서가 상관없는 차이만 존재합니다.
- 6C3을 생각해보면 (1,2,3), (1,2,4), (1,2,5), etc , (4,5,6)을 선택합니다.
- 깊이가 증가하면 , 이전의 원소+1 부터 탐색을 수행합니다.
- 간단한 문제는 1부터 모두 검사하는 방식으로 해도 시간복잡도가 크지 않기 때문에 지금까지 1로 많이 해왔습니다. 하지만 어려운 문제는 **1부터 모두 검사하면 반드시 시간초과가 발생하기 때문에 이전의 원소+1 부터 탐색을 수행합니다.**
- 이전의 원소+1을 쉽게 사용하기 위해 다음 DFS를 수행할 때 매개변수로 넘겨줍니다.

- 조합이 pair OR 구조체일 때 굳이 1개의 값으로 변경해서 조합을 하지 말고 그냥 Pair OR 구조체를 유지합니다.
- DFS로 모든 조합을 찾고 각각의 조합을 찾았을 때 또 다른 탐색문을 수행합니다. **(이중 탐색)**
- <https://www.acmicpc.net/workbook/view/2052>
- 연구소 문제 , 감시 문제(boj_15683)

4. BFS

1) 3가지 핵심 요소

① 자료구조(Queue OR Priority Queue)

② Visit 배열

- 부모에서 자식으로 탐색을 진행할 때 , 이전에 탐색한 자식을 가면 무한루프가 생성되므로 Visit 배열을 통해 검사합니다.

- 문맥적으로 True / False가 옳기 때문에 bool 자료구조가 옳지만 , **가독성 측면에서 1 / 0** 이 보기 좋습니다.

③ **복잡한 구조의 값을 간단한 변수로 변경** : 자료구조를 다루는 코딩이 진행되므로 자료구조를 탐색하는 index를 변수화 시켜서 가독성을 높입니다.

```
while(q.empty() == false)
{
    cur_r = q.front().first;
    cur_c = q.front().second;
    for(int i=0; i<4; i++)
    {
        next_r = cur_r+path[i].first;
        next_c = cur_c+path[i].second;
        if(visited[next_r][next_c] == 0 && limit_l <= abs(arr[cur_r][cur_c] - arr[next_r][next_c]) && abs(arr[cur_r][cur_c] - arr[next_r][next_c]) <= limit_r)
        {
            visited[next_r][next_c] = 1;
            q.push({next_r,next_c});
            v.push_back({next_r,next_c,arr[next_r][next_c]});
            is_move = true;
        }
    }
    q.pop();
}
```

- 기본적으로 캐싱을 하기 때문에 시간 절약의 목적이 가장 큼니다.

- **Queue가 비어 있는 상태가 될 때 까지 반복하여 탐색** 하는 것이 핵심입니다.

- 시작점부터 다른 점까지의 거리를 구하는 문제

2) 시간복잡도

- DFS와 완전히 동일합니다.

① 인접 행렬

= $O(V^2)$

ex) 5*5행렬이면 25개의 공간 생깁니다. (1,1)부터 시작해서 별다른 조건이 없다면 (5,5)까지 모든 행렬의 원소를 검사하기 때문에 V^2

② 인접 리스트

= $O(V+E)$

- 인접리스트는 간단하게 요약하면 BFS할 때 V는 인접 행렬과 동일하지만 각각의 노드마다 인접한 노드를 인접 리스트로 구현했고 이들의 개수는 최대 $2 * E$ 이기 때문에 $V+E$ 입니다.

3) 실수 하는 경우

① Queue.front()

- BFS는 기본적으로 Queue의 front()에 있는 원소부터 Queue가 빌 때 까지 순서대로 반복검사를 하는 기법입니다. 하지만 대다수의 일반 반복문은 index를 순회하므로, **queue.front()를 사용하지 않고 index를 사용하는 실수**를 범할 수 있습니다.
- 실수를 방지하기 위해 Queue.front().first , Queue.front().second를 모두 변수에 저장하는 것이 좋고 , 가독성에도 좋습니다.

② 메모리 초과 (Memory Exceeded)

pjw960316

2 4963

메모리 초과

- Visit 검사를 이상한 위치에서 검사하고 있는지 OR 올바른 원소(queue.front())는 실시간으로 변함)를 검사하고 있는지 **체크**해야 합니다.
- 위의 케이스로 인해 Visit 검사가 제대로 이루어지지 않으면 Queue에 계속 데이터를 push해서 메모리 초과가 발생합니다.
- 어떤 노드를 방문했는지 검사하는 조건문에서 방문을 했다고 결과가 나오면 그 즉시 해당 노드를 방문했다는 코드를 작성합니다.

<Example : 메모리 초과 예제 boj_4963 (섬의 개수)>

reference : <https://technote-mezza.tistory.com/66>

```
if (arr[i][j] == 1 && visit[i][j] == 0)
{
    //visit[i][j] = 1; //쓸모 없는 Visit 검사_1
    q.push({ i,j });
    while (q.empty() == false)
    {
        arr[q.front().first][q.front().second] = island_num;
        //visit[q.front().first][q.front().second] = 1; //쓸모 없는 Visit 검사_2
        for (int k = 0; k < 8; k++)
        {
            cur_r = q.front().first + path[k].first;
            cur_c = q.front().second + path[k].second;
            if (arr[cur_r][cur_c] == 1 && visit[cur_r][cur_c] == 0)
            {
                q.push({ cur_r, cur_c });
                visit[cur_r][cur_c] = 1;
            }
        }
        q.pop();
    }
    island_num++;
}
```

4) 문제 유형

- 최단거리(dijkstra)
- 배열 전체를 순회하면서 특정 값을 만날 때마다 부분적으로 BFS 탐색 : boj_16234번



- 시작점(1,1)부터 시작해서 모든 흰색 블록을 빨간색으로 만들어야 합니다. 단, 검은색으로 칠해진 블록은 지날 수 없습니다.
- 시작점부터 BFS를 1번만 수행하면 (3,3)은 빨간색으로 칠할 수 없습니다.
- 9가지 블록을 모두 검사해서 아직 방문하지 않은 블록(visited[] = 0)이라면 BFS를 하는 방식으로 해결합니다.

5) 번뜩인 비유!

- 제가 4월 계획표(4월 1일부터 4월 30일까지)를 작성한다고 생각합시다. queue에 4월 1일에 할 일들을 모두 적습니다. 그리고 모두 수행합니다. 그 후 queue에서 4월 1일의 할 일들을 모두 제거합니다. queue에 4월 2일에 할 일들을 모두 적습니다. 그리고 모두 수행합니다. 그 후 queue에서 4월 2일의 할 일들을 모두 제거합니다. 이를 30일까지 합니다. 그리고 5월이 되면 4월 계획표는 종료됩니다.
- 4월 1일과 4월 2일을 독립적으로 고려하는 경우가 문제에서 많이 발생할 것 입니다.
- 어불성설이지만 4월 1일에 한 일은 4월 2일에 다시 하지 않습니다. (visit 개념)
- 4월 1일의 일들은 모두 동등한 조건에서 통제되어야 하고, 4월 2일의 일들은 모두 동등한 조건에서 통제되어야 합니다.

<Example>

4월 1일	4월 2일	4월 2일	4월 2일	4월 3일	4월 3일	4월 3일	4월 3일	4월 3일	4월 3일
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

- queue에서 pop을 하지 않는다고 가정하면 이렇게 데이터가 쌓이게 됩니다. 그리고 이렇게 날짜로 구분해서 생각하는 사고가 BFS에서 핵심이 됩니다.
- 자료구조든 데이터든 각각의 일자에 맞게 수행한 일을 저장해야 합니다. 그래서 보통 추가적으로 vector나 array에 데이터를 저장합니다.

<7강_그래프 심화>

1. 최단거리 (Dijkstra Algorithm)

- 임의의 시작 노드에서 모든 노드까지의 최단경로
- 간선의 가중치가 음수인 경우는 해결 할 수 없습니다.
- 궁극의 목표 : 그래프의 모든 노드를 S에 채워 넣기입니다. 다시 말해 , 모든 노드의 최단 경로를 확정 시켜야 합니다.

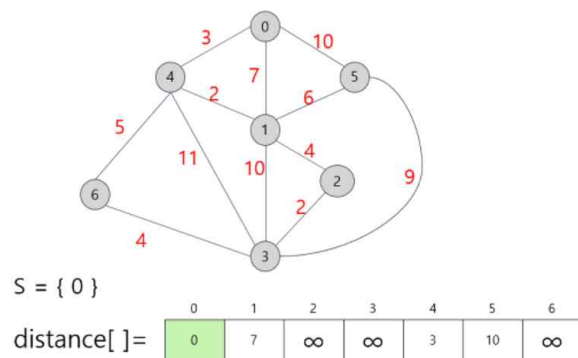
<자료구조>

- S[] : set(확정)입니다. 여기 저장되는 노드는 최종 최단경로를 확정 지었다는 의미입니다.
- distance[] : 시작 노드에서 해당 index(=node번호)까지의 현재까지의 최단경로
- 인접행렬 OR 인접리스트

*** 설명에서는 인접행렬을 사용했다고 가정하겠습니다. ***

*** 임의의 시작 노드를 Node_0 라고 가정하겠습니다. ***

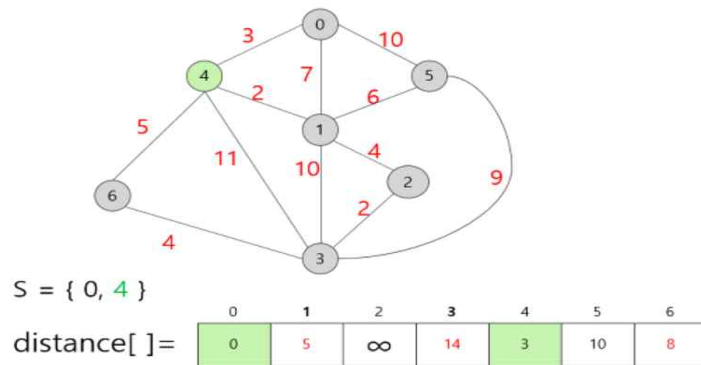
1) 1단계



- 시작한 정점은 default로 S에 추가시킵니다.
- 현재 알고 있는 정보는 인접행렬에서 주어진 인접한 노드와의 거리 밖에 없습니다.
- 이를 전부 distance에 갱신시킵니다.

2) 2단계

- 이 과정은 예시를 통해 설명하겠습니다.



- 모든 과정을 완벽하게 이해합니다.
- 이전에 갱신한 distance[]에서 S에 저장한 노드를 제외한 노드 중에 가장 작은 값을 갖는 노드를 고릅니다. S에 노드_0이 저장되어 있으므로 0을 제외한 노드_4가 distance[]에서 가장 작은 값을 갖기 때문에 노드_4를 고릅니다.
- 노드_4에서 인접한 노드들과의 거리(add_dis)를 각각 계산합니다.
- 기존의 distance[]에는 노드_4까지의 거리(3)가 저장되어 있고, 3과 add_dis를 더해서 새로운 거리(new_dis)를 각각 계산합니다.
- new_dis들과 기존의 distance[]를 비교해서 new_dis가 작으면 갱신합니다.

<그림 설명>

- 노드_4는 노드_0,1,3,6과 인접해 있습니다.

노드	기존 dis[]	3 + add_dis = new_dis
0	0	3 + 3 = 6
1	7	3 + 2 = 5
3	INF	3 + 11 = 14
6	INF	3 + 5 = 8

- 기존 dis[]보다 작은 new_dis는 노드_1,3,6 이므로 그들을 갱신시킵니다.

3) 3단계

- S에 모든 노드를 저장할 때 까지 2단계를 반복합니다.
- 갱신한 distance[]에서 S에 저장한 노드를 제외한 노드 중에 가장 작은 노드(Cur_Node)가 갖는 값은 시작노드인 노드_0에서 Cur_Node까지의 현재까지의 최단경로지만, 이 값이 결국 시작노드인 노드_0에서 Cur_Node까지의 최종 최단경로가 됩니다.
- 공부를 하면서 느낀 것 : distance[]에서 가장 가중치를 갖는 노드를 찾는 과정에서 우선순위 큐를 사용하지 않으면 V^2 이므로 시간복잡도를 넘어 버리는 현상이 발생합니다.

2. 그래프 탐색문제에서의 상태 공간 트리(State-Space Tree)

1) 상태 공간 트리(State-Space Tree)를 머릿속으로만 고려할 뿐 , 실제로 각각의 상태 공간을 저장하지 않아도 되는 문제

문제

N×M크기의 배열로 표현되는 미로가 있다.

1	0	1	1	1	1
1	0	1	0	1	0
1	0	1	0	1	1
1	1	1	0	1	1

미로에서 1은 이동할 수 있는 칸을 나타내고, 0은 이동할 수 없는 칸을 나타낸다. 이러한 미로가 주어졌을 때, (1, 1)에서 출발하여 (N, M)의 위치로 이동할 때 지나야 하는 최소의 칸 수를 구하는 프로그램을 작성하시오. 한 칸에서 다른 칸으로 이동할 때, 서로 인접한 칸으로만 이동할 수 있다.

위의 예에서는 15칸을 지나야 (N, M)의 위치로 이동할 수 있다. 칸을 셀 때에는 시작 위치와 도착 위치도 포함한다.

입력

첫째 줄에 두 정수 N, M($2 \leq N, M \leq 100$)이 주어진다. 다음 N개의 줄에는 M개의 정수로 미로가 주어진다. 각각의 수들은 붙어서 입력으로 주어진다.

출력

첫째 줄에 지나야 하는 최소의 칸 수를 출력한다. 항상 도착위치로 이동할 수 있는 경우만 입력으로 주어진다.

= DFS BFS 문제를 나뉘보면 상태 공간 트리를 그리지 않고 , 현재 위치에서 4방향으로 이동하여 새로 생긴 state가 굳이 4가지의 state-space를 그리지 않아도 되는 문제

- boj_2178

= 미로에서 플레이어가 4방향으로 이동할 때 마다 **미로에 벽이 생기거나 미로가 풀리는 등의 변화가 없기 때문에** 새로운 space state가 필요없음.

- 새롭게 4방향으로 이동시킨 결과 4개의 새로운 state를 트리의 노드로 하여 기록해야 하는 문제

2) 상태 공간 트리(State-Space Tree)를 실제로 자료구조에 저장을 해야 하고 , 더 어려운 문제는 자료구조에 저장하는 자료가 Struct 형태.

- BOJ_14502번 : 자료구조에 저장하는 state가 간단

- BOJ_12100번 : 자료구조에 저장하는 state가 복잡

- 문제에서 주어진 **상태 공간에서 노드가 이동함에 따라 상태 공간의 변화가 발생**하여 변화한 상태공간들을 트리구조로 자료구조에 저장해야 합니다.

- bfs를 할 때 마다 queue에 state-space를 하나의 노드(state)로 저장합니다. -> dfs는 모르겠음

- boj_14502번은 4방향으로 바이러스가 이동할 때 마다 state-space에 바이러스의 개수가 달라지고 , queue에 state-space를 원소로 저장하며 트리의 bfs 탐색을 진행합니다.

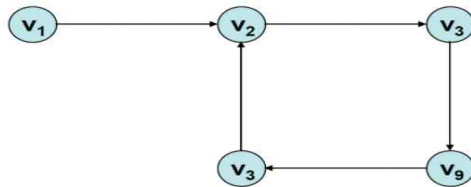
3. 연결(그래프)

connected

임의의 두 노드가 연결되었다(connected)는 것은 두 노드 사이에 경로가 존재한다는 이야기입니다. 이와 관련해 다음과 같이 정의됩니다.

- 모든 노드쌍 사이에 경로가 존재하는 무방향그래프는 연결되었다고 말한다.
- 임의의 방향그래프에서 방향을 무시하고 보면 연결되어 있을 경우, 해당 방향 그래프는 연결되었다고 말한다.
- 방향그래프의 임의의 노드쌍 a, b 에 대해 a 에서 b 로 가는 경로, b 에서 a 로 가는 경로가 존재한다면, 해당 방향그래프는 **강연결 (strongly connected)** 되었다고 말한다.

아래 방향그래프에서 방향을 무시하고 보면 이 그래프 내 임의의 노드쌍 사이에 모두 경로가 존재함을 알 수 있습니다. 따라서 아래 방향그래프는 *connected graph*입니다. 하지만 v_1 에서 v_3 로 가는 경로는 존재(v_1, v_2, v_3, v_9, v_3)하나, 반대로 v_3 에서 v_1 으로 가는 경로는 존재하지 않는다는 점에서 *strongly connected graph*는 아닙니다.



= 그래프에서 임의의 두 정점을 잡았을 때 어떠한 경우에도 두 정점 사이에 경로가 존재할 때 그 그래프를 **연결그래프**라고 부릅니다.

- 완전 그래프와는 잘 구분해야 합니다. 완전 그래프는 임의의 두 정점 사이에 Edge가 존재하는 그래프이고, 연결 그래프는 임의의 두 정점 사이에 경로가 존재하는 그래프입니다.

- **연결 그래프** = 약한 **연결그래프**입니다.(비방향성 그래프에서의 개념) -> 11-4강에서 spanning tree를 공부할 때 나옵니다.

- 아래에서 배울 연결 요소는 연결그래프와 동등하게 생각해도 괜찮은 것 같습니다.(확실 X)

4. 연결 요소

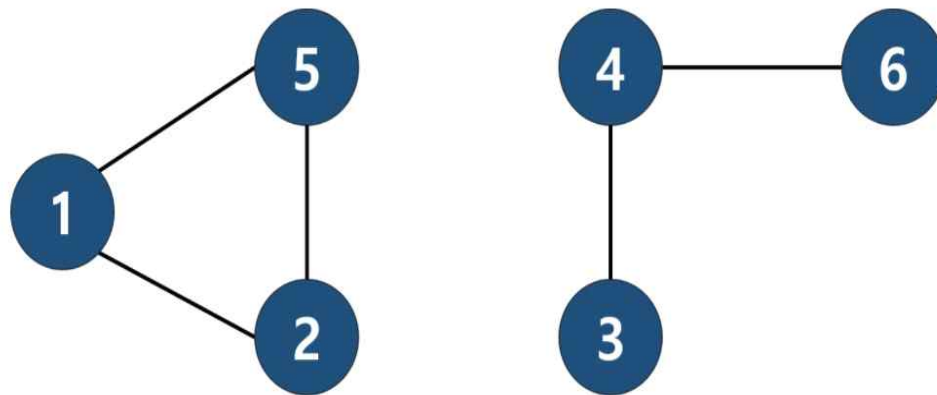
- 정점 1개가 혼자 떨어져 있어도 연결요소 입니다.

1) (약한) 연결 요소(Connected Component)

- **비방향성 그래프**에서의 연결 요소 개념입니다.
- 문제에서 방향성 그래프로 주어진다면 , 방향을 제거하여 비방향성 그래프로 생각하고 (약한) 연결 요소를 구합니다.

그래프 G의 연결 요소는 G의 다른 연결 부
그래프의 진 부그래프가 아닌 G의 연결 부
그래프이다.

즉, G의 연결요소는 G의 최대 연결 부그래
프이다.



- 6개의 노드로 이루어진 그래프를 다음과 같이 2개의 Sub-Graph로 나눌 수 있습니다.
- 결론부터 말하면 각각의 Sub-Graph({1,2,5}, {3,4,6})는 연결 요소 입니다.
- 하나의 연결요소에서 모든 node 끼리는 경로가 존재해야 합니다.
- 서로 다른 연결요소의 node 끼리는 edge가 존재하면 안 됩니다.
- 그래프에서 연결요소는 Disjoint-Set(=Union-Find)으로 찾을 수 있습니다.

2) 강한 연결 요소(Strongly Connected Component)

- 방향성 그래프에서의 연결 요소 개념입니다.
- 방향성 그래프에서 모든 정점이 모든 다른 정점에 대해 도달 가능한 경우입니다.

3) 그래프를 연결하는 방법

<절단 정점>

= 하나의 정점을 없앴을 때 그래프가 분할(연결 요소 증가)되는 정점

- 정점과 연결된 모든 edge를 삭제합니다.
- 언제나 모든 그래프가 절단 정점을 갖지는 않습니다.

ex) 완전 그래프(K_n)

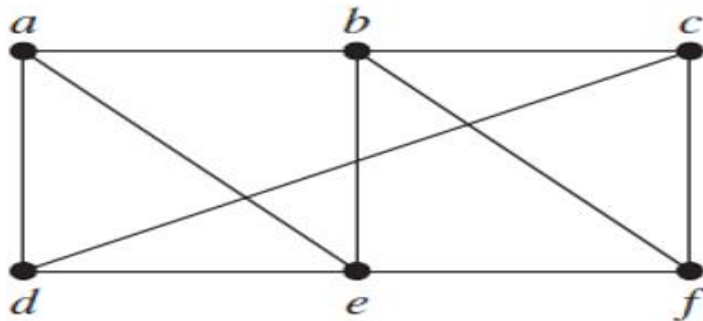
- 정점을 없애는 개념이기 때문에 혼자 독립적으로 존재하는 연결요소가 되지 않습니다.

ex) 정점 a를 없애면 정점 a가 혼자 따로 존재해서 연결요소가 되는 것이 아니라 그래프 세상에서 아예 사라지는 것입니다.

<절단 모서리>

= 하나의 모서리를 없앴을 때 그래프가 분할(연결 요소 증가)되는 모서리

<정점 절단>



- 절단 정점은 정점 1개를 없애는 것이라면, 정점 절단은 정점 여러 개를 없애는 것 입니다.
- 완전 그래프를 제외한 모든 그래프는 정점 절단이 존재합니다.
- 최소 개수의 정점들을 없애서 분할했을 때 정점의 개수를 카파(k)라고 합니다.
- b, c, e를 없애면 그래프가 분할됩니다.

<8강_구현과 문자열>

1. 구현 문제의 특징

- 흔히 알고리즘 대회에서 구현 유형의 문제란 무엇을 의미할까요?
 - 풀이를 떠올리는 것은 쉽지만 소스코드로 옮기기 어려운 문제를 지칭합니다.
 - 구현 유형의 예시는 다음과 같습니다.
 - 알고리즘은 간단한데 코드가 지나칠 만큼 길어지는 문제
 - 실수 연산을 다루고, 특정 소수점 자리까지 출력해야 하는 문제
 - 문자열을 특정한 기준에 따라서 끊어 처리해야 하는 문제
 - 적절한 라이브러리를 찾아서 사용해야 하는 문제
- 문제를 풀 로직만 보면 간단해보여서 무작정 코딩을 시작하다가 말리는 문제
- 2차원 배열(벡터)을 이용하는 문제가 많이 출제됩니다.

2. NULL vs NUL vs '\0'

1) NUL vs '\0'

- NUL 과 '\0'은 같습니다.

NUL과 '\0' 비교

NUL과 '\0'은 같다.

ASCII Code표의 매핑 규칙에 따르면 0을 문자로 표기한 것이 NUL 문자다.

	NUL	'\0'
Definition	- 문자상수 - ASCII Code의 첫 번째 문자 0	
Data Type	char	
Type Conversion	동일한 타입이다.	
Considerations	문자열은 항상 NUL로 끝나야 한다. NUL을 만나기 전까지의 문자들을 문자열이라고 한다.	

2) NULL vs NUL(='\0')

NULL, 0 비교

NULL은 널 포인터로서, (void *)0을 가리킨다.

NULL 매크로는 <stdio.h>, <stdlib.h> 등 여러 헤더파일에서 이미 정의돼있다.

C언어에서 pointer와 정수는 서로 형 변환될 수 있는데, 두 데이터 타입 모두 4byte다.

Null pointer는 정수 0으로 변환될 수 있고, 정수 0은 Null pointer로 변환될 수 있다.

즉, Null pointer는 메모리 주소 0을 가리키는 pointer라는 뜻이다. 0번지는 일반적으로 접근할 수 없는 메모리 영역이다.

따라서 pointer가 아무것도 가르키지 않을 때 Null pointer로 초기화 한다.

	NULL	0
Definition	- 널 포인터 0x00000000 - '값이 없다', '비어있다'를 의미한다. - 정의되었지만 가리키는 것이 없다.	정수 0

```
cout << sizeof(NULL) << " , " << sizeof('\0'); //4 , 1
```

- 4바이트를 모두 0으로 채운 값은 접근할 수 없는 메모리 영역(포인터 주소 값)입니다.
- 그래서 NULL에 해당 값을 채워 넣었고, NULL = NULL Pointer = 0x00000000을 의미합니다.
- 0x00000000은 결국 정수 0 입니다.
- NULL 과 '\0'은 문자열과 문자의 비교이므로 다르게 생각하는 것 이 옳습니다.

<Null pointer>

= 포인터는 해당 데이터가 저장 되어 있는 가상 주소를 저장합니다. 하지만 가상 주소 이외에 NULL값을 정의할 수 있습니다. **포인터에 NULL값을 저장하면 해당 포인터가 아무것도 가리키지 않는다는 것**을 의미합니다.

ex) 'int *ptr' 과 'int *ptr = NULL'에서 ptr은 모두 Null Pointer 입니다.

- 보통 pointer는 가상 주소로 이동하여 해당 값을 참조하는데 , Null pointer는 저장한 가상 주소가 없기 때문에 역 참조(포인터가 가리키는 가상 메모리의 데이터가 저장한 값을 참조)를 요청하면 그 유명한 Null Pointer Exception이 발생합니다.

<Null Pointer Exception>

```
//1. 정상적인 포인터
int a = 77;
int *ptr = &a;
printf("%d" , *ptr);

//2. NULL-Pointer에서 발생하는 2가지 Segmentation Fault
int *ptr_null; //same as int *nullptr = NULL;

//1) 널 포인터에 어떤 값 할당하기.
*ptr_null = 7;

//2) 널 포인터가 가리키는(사실은 아무것도 가리키지 않음) 변수 값 출력.
printf("%d" , *ptr_null);
```

① 1번 : ptr이 참조하는 변수의 값인 77이 정상적으로 출력됩니다.

② 2번 : ptr_null은 아무것도 참조하지 않는 NULL Pointer 입니다. 다음 2가지 Case는 Null Pointer에서 Segmentation Fault를 발생시키는 대표적인 Null Pointer Exception 입니다.

<case_1> : Null Pointer에 어떤 값을 할당하도록 명령을 내리면 Null Pointer Exception이 발생합니다.

<case_2> : Null Pointer는 아무 것도 참조하지 않는데 Null Pointer가 참조하는 변수의 값을 출력하도록 명령을 내리면 Null Pointer Exception 이 발생합니다.

4) 문자열 마지막의 '\0'

= 아스키코드표 0번의 NUL입니다.

ASCII Table - 아스키 코드표입니다.

제어 문자		공백 문자		구두점		숫자		알파벳			
10진	16진	문자	10진	16진	문자	10진	16진	문자	10진	16진	문자
0	0x00	NUL	32	0x20	SP	64	0x40	@	96	0x60	
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	\$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	HT	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t

- 1byte의 char형 데이터가 모두 0으로 채워진 문자가 NUL , 다시 말해 '\0'을 의미합니다.
- 문자열에의 마지막에 '\0'을 적어 주어 문자열의 종료를 나타냅니다.
- char arr[10]으로 선언하여도 모두 0x00으로 초기화 되지만 문자열을 다루다보면 char arr[10] = "\0"로 선언해주는 것이 더욱 안전했습니다.

```

1 #include <stdio.h>
2
3 int main()
4 {
5     char a[6] = "abc";
6
7     for(int i=0; i<6; i++)
8     {
9         printf("0x%02x  ", a[i]);
10    }
11    return 0;
12 }

```

```

User@DESKTOP-BAGIJS2 ~
$ ./a.exe
0x61  0x62  0x63  0x00  0x00  0x00

```

= 문자열 배열에 남은 메모리 공간은 0x00(='\0')으로 채워줍니다.

- 문자 '1'은 %c에서는 1이지만 , %d에서는 49입니다. 우리는 0x49를 '1'로 하기로 약속했습니다.
- ' '로도 인식하지만 0x20 입니다.

3. char[]문자열은 '\0'이 맨 마지막에 존재하는데, C++ String도 그런가?

기존 C 언어에서는 문자의 나열과 마지막에 '\0' 을 넣어 문자열을 표현합니다.

그에 반해 `std::string` 은 문자의 나열과 길이로 문자열을 표현합니다. 따라서 마지막에 '\0' 을 넣지 않아도 문자열의 끝을 알 수 있습니다.

예를들어 "hello" 를 `std::string` 변수에 넣는다면, 문자열의 길이인 5와 문자들인 (h, e, l, l, o) 가 저장됩니다. 따라서 `operator[]` 를 통해 인덱스 5에 접근하려 한다면 문자열의 범위를 넘어서기 때문에 `std::out_of_range` 예외를 발생합니다. 참고로 `operator[]` 에서 예외 발생은 표준 사항이 아니기 때문에 발생하지 않을 수도 있습니다.

하지만 실제로 `std::string` 은 `NULL('\0')` 로 끝나는 문자열과의 호환성을 위해서 메모리 상에 '\0' 을 포함하여 문자열을 저장합니다. 다만 `operator[]` 를 통한 '\0' 에 대한 접근을 허용하지 않을 뿐입니다. 그리고 이 때 저장되는 '\0' 은 호환용일 뿐 문자열의 길이를 계산 한다거나 다른 용도로는 사용되지 않습니다.

따라서 문자열의 끝을 확인하기 위해서 '\0' 검사를 하실 필요가 없습니다. 이미 문자열의 길이는 저장되어 `length()` 또는 `size()` 를 통해 가져올 수 있기 때문입니다.

- 결과적으로, string 끝에는 '\0'이 존재하지 않는다고 생각해도 무방합니다.

3. 문자열을 어떠한 문자를 기준으로 분리하여 문자열 벡터 형성하는 방법

```
vector<string> tokenize_getline(const string& data, const char delimiter = ' ') {  
    vector<string> result;  
    string token;  
    stringstream ss(data);  
  
    while (getline(ss, token, delimiter)) {  
        result.push_back(token);  
    }  
    return result;  
}
```

4. 문자열 분리 : substr

```
#include <iostream>  
#include <string>  
  
int main()  
{  
    std::string numbers = "0123456789";  
    std::string full = numbers.substr();  
    std::string sub = numbers.substr(3, 5);  
  
    std::cout << "Full String: " << full << std::endl;  
    std::cout << "Sub String: " << sub << std::endl;  
}
```

아무것도 입력하지 않고 호출하는 경우 동일한 문자열이 새로운 메모리에 할당되어 리턴됩니다.

시작 지점과 길이를 전달하면 인덱스의 시작부터 길이만큼의 문자열을 출력합니다.

```
Full String: 0123456789  
Sub String: 34567
```

실행 결과

```
string original_str = "abcdef";  
string divided_str = "";  
divided_str = original_str.substr(시작 index , 개수)
```

- 좌측의 예시인 (3,5)는 3번 index부터 5개의 문자를 읽어 와서 문자열로 형성한다는 의미입니다.

5. C++ string은 포인터 타입이 아니다!

No. `std::string` is not a pointer type; it cannot be made "null." It cannot represent the absence of a value, which is what a null pointer is used to represent.

It can be made *empty*, by assigning an empty string to it (`s = ""` or `s = std::string()`) or by clearing it (`s.clear()`).

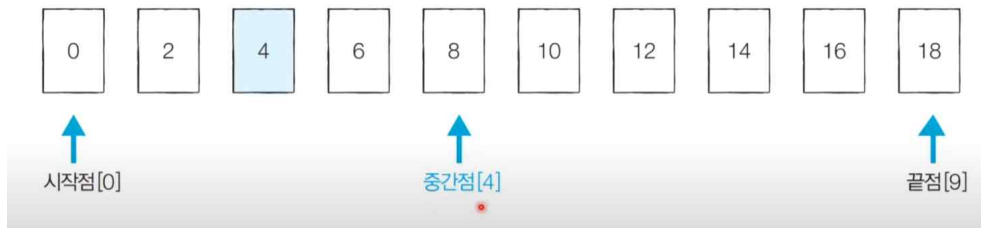
```
string str = NULL;
```

- 포인터 타입이 아니기 때문에 위의 코드처럼 NULL로 초기화가 되지 않습니다.
- string을 빈 문자열로 초기화 하려면 `string str = ""`처럼 초기화 합니다.

<9강_탐색>

1. 이진 탐색

- 이진 탐색을 시도하려면 해당 자료구조는 반드시 정렬되어 있어야 합니다.
- $O(\log N)$ 의 시간 복잡도를 갖습니다.
- Index를 갖고 이진 탐색을 하는 것이지 값으로 하는 것은 아닙니다.
- Start는 0번 Index로 시작하고 , End는 마지막 Index(자료구조 size - 1)로 시작합니다.
 - [Step 1] 시작점: 0, 끝점: 9, 중간점: 4 (소수점 이하 제거)



- middle 값이 항상 헛갈렸는데 , 언제나 $(start + end)/2$ 로 계산하고 소수점은 버립니다.

2. 이진 탐색과 Set

<부록_기타 공부 자료>

1. Struct의 주소와 크기

```
1 #include <stdio.h>
2
3 struct str
4 {
5     int x;
6     char y;
7     int z;
8 };
9
10 int main()
11 {
12     struct str obj;
13     struct str* ptr = &obj;
14     printf("member value's address : %x %x %x\n", &obj.x, &obj.y, &obj.z);
15     printf("object's address : %x\n", &obj);
16     printf("pointer which point obj : %x\n", ptr);
17     //printf("struct's address : %x\n", &str); -> ERROR : struct doesn't have address
18     return 0;
19 }
20
```

```
User@DESKTOP-BAGIJS2 ~
$ gcc str.c

User@DESKTOP-BAGIJS2 ~
$ ./a.exe
member value's address : fffffcc2c fffffcc30 fffffcc34
object's address : fffffcc2c
pointer which point obj : fffffcc2c

User@DESKTOP-BAGIJS2 ~
$ |
```

- 구조체는 메모리에 저장되지 않습니다. 그러므로 주소가 존재하지 않습니다.
- 구조체 포인터도 결국은 구조체로 만든 객체의 주소를 저장합니다.
- 객체는 메모리의 스택영역에 저장됩니다. 객체의 멤버변수들이 순서대로 메모리에 저장됩니다.
- char형은 1byte이지만 구조체에서 가장 큰 자료형이 int형이므로 padding이 발생하여 4byte로 저장됩니다.
- C언어도 객체지향 프로그래밍처럼 구조체 내부에 함수를 어려운 방법을 통해 만들 수는 있지만 기본적으로는 구조체 내부에 멤버함수를 만들 수 없습니다.

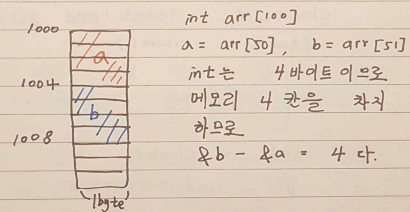
2. 메모리 할당(Int, Double, Char)

char, int, double 값 저장 원리

컴퓨터마다 1 word = $\bigcirc$ byte 의 값이 다르다.
우린 4 라고 하자

1. int

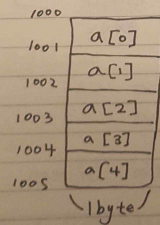
int는 4 byte 이고 메모리에
아래와 같이 저장된다.



2. double

→ int와 같지만 byte는 8 바이트

3. char



한글은 ASCII가 아니라 1 byte 표현이
안되서 2 byte 이다.

char는 1 바이트이다.
∴ 0 ~ 2⁸-1 저장 가능
해서 ASCII가
255(2⁸-1) 까지 다.

∴ ASCII는 문자이고.
문자 0이 int로 48.

char str[5] = "jmwon"

'n'과 '\0'의 주소 차이는 1.

3. if-else vs switch-case

1) switch-case문 예제

```
int n = 777;
switch(n)
{
    case 777 :
    {
        cout << 777;
        break;
    }
    case 34 :
    {
        cout << 34;
        break;
    }
    default :
    {
        cout << "default";
    }
}
```

- n에 들어갈 수 있는 type은 char OR int입니다. double형은 사용할 수 없습니다.
- default는 통상적으로 마지막에 들어가므로 break를 적지 않습니다. 실수로 default를 가운데에 적으면 예상하지 못한 ERROR가 발생할 수 있으므로 반드시 마지막에 사용하는 것이 안전합니다.
- default는 if-else의 else 이므로 필요 없다면 사용하지 않아도 무방합니다.

2) 상황별 사용

- ① 5회 이하의 분기가 발생하면 if-else
- ② 5회 초과인 분기가 발생하면 switch-case

3) switch-case의 장점

- ① 5개 초과인 분기가 발생하는 시점부터 속도가 빨라집니다.
- ② 가독성에 좋습니다.

<https://www.geeksforgeeks.org/switch-vs-else/>

4. Segmentation Fault (core dumped) 디버깅 -> 실전 예제도 추가하고 싶음.

1) 컴파일 옵션으로 -g를 추가하여 컴파일을 합니다.

- 옵션 -g를 사용하지 않으면 gdb 결과 창에서 어떤 줄에서 Segmentation Fault가 발생했는지 알 수 없습니다.

```
pjw96@ParkJiWon ~/B0J/source  
$ g++ -g test.cpp
```

2) gdb 열기

- gdb에서 실행 파일을 엽니다.

```
pjw96@ParkJiWon ~/B0J/source  
$ gdb ./a.exe
```

3) gdb 에서 디버깅

① 명령어 run : 실행 파일을 gdb 내에서 실행시킵니다.

② bt : Segmentation Fault가 발생한 위치만 표기해줍니다.

```
(gdb) run  
Starting program: /cygdrive/c/Users/pjw96/OneDrive/바탕 화면/Cygwin Source/B0J/source/a.exe  
[New Thread 14776.0x3ac4]  
[New Thread 14776.0x1914]  
[New Thread 14776.0x2ba8]  
[New Thread 14776.0x2e54]  
[New Thread 14776.0x3bb8]  
  
Thread 1 "a" received signal SIGSEGV, Segmentation fault.  
0x00000001004010bf in main () at test.cpp:36  
36      *(str+1) = 'n';  
(gdb) bt  
#0  0x00000001004010bf in main () at test.cpp:36
```

5. enum

```
enum num
{
    ONE = 1 , TWO = 2 , THREE = 3 , FOUR = 4
};

int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout << ONE << TWO << THREE << FOUR;
```

```
$ ./a.exe
1234
```

- enum은 기본적으로 상수를 많이 쓸 때 이므로 가로로 길게 쓴다.
- struct와 거의 동일한 방식이라고 보면 됩니다.